EBook Gratis

APRENDIZAJE kubernetes

Free unaffiliated eBook created from
Stack Overflow contributors.

#kubernetes

Tabla de contenido

Acerca de 1

Capítulo 1: Empezando con kubernetes 2

Observaciones 2

Versiones 3

Examples 3

Instalación de minikube 3

Requerimientos 3

Instalación 3

Uso 4

Instalar en Google Cloud 5

Configurar kubectl 5

Google Cloud (motor de contenedores) 5

Minikube 6

Kubectl en linea de comando 6

Hola Mundo 7

Capítulo 2: Kubernetes en producción 10

Introducción 10

Examples 10

Implementar el clúster zookeeper en producción usando kubernetes y ceph 10

Capítulo 3: Llamando a la API de Kubernetes 14

Examples 14

Usando Kubernetes Go Client - Fuera de Cluster 14

Listar conjuntos de réplicas por implementación dada con kubernetes go client 14

Retroceder con la revisión de conjuntos de réplicas usando kubernetes go client 16

Actualización continua con repliasets utilizando kubernetes go client 17

Uso de Kubernetes Go Client - Dentro de Cluster 19

Creditos 20

Acerca de

You can share this PDF with anyone you feel could benefit from it, downloaded the latest version from: kubernetes

It is an unofficial and free kubernetes ebook created for educational purposes. All the content is extracted from Stack Overflow Documentation, which is written by many hardworking individuals at Stack Overflow. It is neither affiliated with Stack Overflow nor official kubernetes.

The content is released under Creative Commons BY-SA, and the list of contributors to each chapter are provided in the credits section at the end of this book. Images may be copyright of their respective owners unless otherwise specified. All trademarks and registered trademarks are the property of their respective company owners.

Use the content presented in this book at your own risk; it is not guaranteed to be correct nor accurate, please send your feedback and corrections to info@zzzprojects.com

Capítulo 1: Empezando con kubernetes

Observaciones

Kubernetes es una plataforma de código abierto para automatizar la implementación, escalado y operaciones de contenedores de aplicaciones en grupos de hosts, proporcionando infraestructura centrada en contenedores.

Con Kubernetes, puede responder rápida y eficientemente a la demanda de los clientes:

- Despliegue sus aplicaciones de forma rápida y predecible.
- Escale sus aplicaciones sobre la marcha.
- Despliegue a la perfección nuevas funcionalidades.
- Optimice el uso de su hardware utilizando solo los recursos que necesita.

¿Por qué necesito Kubernetes y qué puede hacer?

Kubernetes puede programar y ejecutar contenedores de aplicaciones en grupos de máquinas físicas o virtuales. Sin embargo, Kubernetes también permite a los desarrolladores "cortar el cable" a máquinas físicas y virtuales, pasando de una infraestructura **centrada en el host** a una infraestructura **centrada** en el **contenedor** , que proporciona todas las ventajas y beneficios inherentes a los contenedores. Kubernetes proporciona la infraestructura para construir un entorno de desarrollo verdaderamente centrado en contenedores.

Kubernetes satisface una serie de necesidades comunes de aplicaciones que se ejecutan en producción, como:

- ubicar conjuntamente los procesos de ayuda, facilitar las aplicaciones compuestas y preservar el modelo de una aplicación por contenedor,
- montaje de sistemas de almacenamiento,
- distribuyendo secretos,
- control de salud de la aplicación,
- replicando instancias de aplicaciones,
- autoescalamiento horizontal,
- nombramiento y descubrimiento,
- balanceo de carga,
- actualizaciones continuas,
- monitoreo de recursos,
- registro de acceso e ingestión,
- soporte para introspección y depuración, y
- Identidad y autorización.

Esto proporciona la simplicidad de la Plataforma como un Servicio (PaaS) con la flexibilidad de la Infraestructura como un Servicio (IaaS), y facilita la portabilidad entre los proveedores de infraestructura.

Versiones

Versión	Fecha de lanzamiento
1.7	2017-06-28
1.6	2017-02-22
1.5	2016-12-13
1.4	2016-09-26
1.3	2016-07-06
1.2	2016-03-17
1.1	2015-09-09
1.0	2015-07-18

Examples

Instalación de minikube

Minikube crea un grupo local de máquinas virtuales para ejecutar Kubernetes. Es el método más sencillo para ensuciarse las manos con Kubernetes en su máquina local.

La documentación para Minikube se puede encontrar en http://kubernetes.io/docs/getting-started-guides/minikube/

Requerimientos

- En macOS, controladores xhyve , VirtualBox o VMware Fusion hipervisores
- En los hipervisores Linux, VirtualBox o KVM.
- En Windows VirtualBox o Hypervisors Hyper-V
- VT-x / AMD-v virtualización habilitada

Para verificar si el soporte de virtualización está habilitado, ejecute el comando apropiado desde abajo. El comando producirá algo si la virtualización está habilitada.

```
# On Linux
cat /proc/cpuinfo | grep 'vmx\|svm'
# On OSX
sysctl -a | grep machdep.cpu.features | grep VMX
```

Instalación

Minikube es un solo binario. Por lo tanto, la instalación es tan fácil como descargar el binario y colocarlo en su ruta.

```
# Specify the version of minikube to download.
# Latest version can be retrieved from
# https://github.com/kubernetes/minikube/releases
VERSION=v0.16.0

# If on Linux
OS=linux
# If on OSX
# OS=darwin

# URL to download minikube binary from
URL=https://storage.googleapis.com/minikube/releases/$VERSION/minikube-$OS-amd64

# Download binary and place in path.
curl -Lo minikube $URL
chmod +x minikube
sudo mv minikube /usr/local/bin/
```

Uso

Para iniciar un nuevo cluster:

```
minikube start
```

Esto creará un nuevo grupo de máquinas virtuales locales con Kubernetes ya instalado y configurado.

Puedes acceder al panel de Kubernetes con:

```
minikube dashboard
```

Minikube crea un contexto relacionado para `kubectl` que se puede usar con:

```
kubectl config use-context minikube
```

Una vez listos, los Kubernetes locales pueden ser utilizados:

```
kubectl run hello-minikube --image=gcr.io/google_containers/echoserver:1.4 --port=8080
kubectl expose deployment hello-minikube --type=NodePort
curl $(minikube service hello-minikube --url)
```

Para detener el clúster local:

```
minikube stop
```

Para eliminar el clúster local, observe que se asignará una nueva IP después de la creación:

```
minikube delete
```

Instalar en Google Cloud

Kubernetes fue desarrollado originalmente por Google para impulsar su motor de contenedores . Como tal, los grupos de Kubernetes son ciudadanos de primera clase en Google.

La creación de un clúster Kubernetes en el motor de contenedor requiere el comando `gcloud` del Google Cloud SDK . Para instalar este comando localmente, use una de las siguientes opciones:

- use el instalador interactivo (la forma más fácil para los recién llegados):

```
curl https://sdk.cloud.google.com | bash
exec -l $SHELL
gcloud init
```

- descargue el SDK desde https://cloud.google.com/sdk/ y ejecute el archivo de instalación apropiado.

  Por ejemplo, para instalar en Linux (x86_64):

```
curl -Lo gcloud-sdk.tar.gz https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/google-cloud-sdk-142.0.0-linux-x86_64.tar.gz
tar xvf ./gcloud-sdk.tar.gz
./google-cloud-sdk/install.sh
gcloud init
```

Una vez que se `gcloud` instalado `gcloud` , cree un clúster Kubernetes con:

```
# Give our cluster a name
CLUSTER_NAME=example-cluster

# Number of machines in the cluster.
NUM_NODES=3

gcloud container clusters create $CLUSTER_NAME --num_nodes=$NUM_VMS
```

Configurar kubectl

Un clúster Kubernetes se controla mediante el comando `kubectl` . El método de configuración de `kubectl` depende de dónde esté instalado Kubernetes.

Google Cloud (motor de contenedores)

Para instalar kubectl utilizando el SDK de Google Cloud:

```
gcloud components install kubectl
```

Para configurar kubectl para controlar un clúster Kubernetes existente en Container Engine:

```
gcloud container clusters get-credentials $CLUSTER_NAME
```

Minikube

Cuando se utiliza minikube, el binario kubectl debe descargarse manualmente y colocarse en la ruta.

```
# Version of Kubernetes.
K8S_VERSION=$(curl -sS https://storage.googleapis.com/kubernetes-release/release/stable.txt)
# Operating System. Can be one of {linux, darwin}
GOOS=linux
# Architecture. Can be one of {386, amd64, arm64, ppc641e}
GOARCH=amd64

# Download and place in path.
curl -Lo kubectl http://storage.googleapis.com/kubernetes-
release/release/${K8S_VERSION}/bin/${GOOS}/${GOARCH}/kubectl
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
```

El binario de minikube configura automáticamente kubectl al iniciar un clúster.

```
minikube start
# kubectl is now ready to use!
```

Kubectl en linea de comando

Después de tener un clúster en ejecución, puede administrarlo con el comando `kubectl` . La mayoría de los comandos se pueden obtener con el comando `kubectl --help` , pero le muestro los comandos más comunes, para administrar y obtener información sobre su cluster, nodos, pods, servicios y etiquetas.

Para obtener información sobre el clúster puede utilizar el siguiente comando

```
kubectl cluster-info
```

Le mostrará la dirección de ejecución y el puerto.

Para obtener información breve sobre los nodos, pods, servicios, etc. o cualquier recurso que tenga lugar en el clúster, puede utilizar el siguiente comando

```
kubectl get {nodes, pods, services, ...}
```

La salida es principalmente una línea por recurso.

Para obtener una descripción detallada de los recursos, puede usar la bandera de `describe` para `kubectl`

```
kubectl describe {nodes, pods, ...}
```

Las aplicaciones implementadas solo son visibles dentro del clúster, por lo que si desea obtener la salida desde fuera del clúster, debe crear una ruta entre el terminal y el clúster kubernetes.

```
kubectl proxy
```

Se abrirá una API, donde podemos obtener todo desde el clúster. Si desea obtener el nombre de los pods para obtener información, debe usar el siguiente comando:

```
kubectl get pods -o go-template --template '{{range .items}}{{.metadata.name}}{{"\n"}}{{end}}'
```

Se enumerarán las vainas para su uso posterior.

```
curl http://localhost:8001/api/v1/proxy/namespaces/default/pods/{pod_name}/
```

Otros dos comandos comunes son los registros de obtención y la ejecución de un comando desde / en la aplicación en contenedor.

```
kubectl logs {pod_name}
kubectl exec {pod_name} {command}
```

La configuración de la finalización de la pestaña para su shell se puede hacer con:

```
source <(kubectl completion zsh)   # if you're using zsh
source <(kubectl completion bash)  # if you're using bash
```

o más programáticamente:

```
source <(kubectl completion "${0/-/}")
```

Hola Mundo

Una vez que su clúster Kubernetes se esté ejecutando y `kubectl` esté configurado, podría ejecutar su primera aplicación en unos pocos pasos. Esto se puede hacer usando los comandos imperativos que no necesitan archivos de configuración.

Para ejecutar una aplicación, debe proporcionar un nombre de implementación (`bootcamp`), la ubicación de la imagen del contenedor (`docker.io/jocatalin/kubernetes-bootcamp:v1`) y el puerto (`8080`)

```
$ kubectl run bootcamp --image=docker.io/jocatalin/kubernetes-bootcamp:v1 --port=8080
```

Confirme que funcionó con:

```
$ kubectl get deployments
NAME                  DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
bootcamp               1         1         1            1         6s
```

Para exponer su aplicación y hacerla accesible desde la ejecución externa:

```
$ kubectl expose deployment/bootcamp --type="LoadBalancer" --port 8080
```

Confirme que funcionó con:

```
$ kubectl get services
NAME                  CLUSTER-IP   EXTERNAL-IP         PORT(S)    AGE
kubernetes            10.0.0.1     <none>              443/TCP    3m
bootcamp              10.3.245.61  104.155.111.170     8080:32452/TCP   2m
```

Para acceder a los servicios, use la IP externa y el puerto de la aplicación, por ejemplo, así:

```
$ export EXTERNAL_IP=$(kubectl get service bootcamp --
output=jsonpath='{.status.loadBalancer.ingress[0].ip}')
$ export PORT=$(kubectl get services --output=jsonpath='{.items[0].spec.ports[0].port}')
$ curl "$EXTERNAL_IP:$PORT"
Hello Kubernetes bootcamp! | Running on: bootcamp-390780338-2fhnk | v=1
```

Lo mismo se podría hacer manualmente con los datos proporcionados en:

```
$ kubectl describe service bootcamp
Name:                   bootcamp
Namespace:              default
Labels:                 run=bootcamp
Selector:               run=bootcamp
Type:                   LoadBalancer
IP:                     10.3.245.61
LoadBalancer Ingress:   104.155.111.170
Port:                   <unset> 8080/TCP
NodePort:               <unset> 32452/TCP
Endpoints:              10.0.0.3:8080
... events and details left out ....

$ export NODE=104.155.111.170
$ export PORT=8080
```

Una vez que esto funcione, puedes escalar tu aplicación con:

```
$ kubectl scale deployments/bootcamp --replicas=4
```

Y verifica el resultado con:

```
$ kubectl get deployments
NAME                  DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
bootcamp               4         4         4            4          30s

$ curl "$EXTERNAL_IP:$PORT"
Hello Kubernetes bootcamp! | Running on: bootcamp-390780338-2fhnk | v=1
$ curl "$EXTERNAL_IP:$PORT"
Hello Kubernetes bootcamp! | Running on: bootcamp-390780338-gmtv5 | v=1
```

Cuidado con el cambio de id de pod.

Para poder ejecutar una nueva versión de la aplicación, ejecute:

```
kubectl set image deployments/bootcamp bootcamp=jocatalin/kubernetes-bootcamp:v2
```

Y confirmarlo con:

```
$ curl "$EXTERNAL_IP:$PORT"
Hello Kubernetes bootcamp! | Running on: bootcamp-284539476-gafwev3 | v=2
```

La limpieza se realiza finalmente con:

```
$ kubectl delete deployment bootcamp
$ kubectl delete service bootcamp
```

Lea Empezando con kubernetes en línea:
https://riptutorial.com/es/kubernetes/topic/4099/empezando-con-kubernetes

Capítulo 2: Kubernetes en producción

Introducción

Presentar cómo utilizar los kubernetes en el entorno de producción.

Examples

Implementar el clúster zookeeper en producción usando kubernetes y ceph

Dockerize zookeeper-3.4.6

Crear un Dockerfile:

```
#######################################################
# Image: img.reg.3g:15000/zookeeper:3.4.6
#######################################################

FROM img.reg.3g:15000/jdk:1.7.0_67

MAINTAINER lth9739@gmail.com

USER root

ENV ZOOKEEPER_VERSION 3.4.6

ADD Dockerfile /

ADD zookeeper/ /opt/

COPY zoo.cfg /opt/zookeeper/conf/zoo.cfg

RUN mkdir -p /opt/zookeeper/{data,log}

WORKDIR /opt/zookeeper

VOLUME ["/opt/zookeeper/conf", "/opt/zookeeper/data", "/opt/zookeeper/log"]

COPY config-and-run.sh /opt/zookeeper/bin/

EXPOSE 2181 2888 3888

CMD ["/opt/zookeeper/bin/config-and-run.sh"]
```

Ver mas detalles

Implementar el controlador de réplica del zookeeper en el clúster kubernetes

---

Puede usar este comando para implementar el controlador de réplica de zookeeper:

```
kubectl create -f zookeeper-rc-1.json
```

```
{
  "apiVersion": "v1",
  "kind": "ReplicationController",
  "metadata": {
    "labels": {
      "component": "zookeeper"
    },
    "name": "zookeeper-1"
  },
  "spec": {
    "replicas": 1,
    "selector": {
      "server-id": "1",
      "role": "zookeeper-1"
    },
    "template": {
      "metadata": {
        "labels": {
          "server-id": "1",
          "role": "zookeeper-1"
        },
        "name": "zookeeper-1"
      },
      "spec": {
        "containers": [
          {
            "env": [
              {
                "value": "1",
                "name": "SERVER_ID"
              },
              {
                "value": "5",
                "name": "MAX_SERVERS"
              }
            ],
            "image": "img.reg.3g:15000/fabric8/zookeeper:latest",
            "name": "zookeeper-1",
            "ports": [
              {
                "containerPort": 2181,
                "name": "client",
                "protocol": "TCP"
              },
              {
                "containerPort": 2888,
                "name": "followers",
                "protocol": "TCP"
              },
              {
                "containerPort": 3888,
                "name": "election",
                "protocol": "TCP"
              }
            ],
            "volumeMounts": [
              {
                "mountPath": "/opt/zookeeper/data",
                "name": "zookeeper-1"
              }
            ]
```

```
          }
        ],
        "restartPolicy": "Always",
        "volumes": [
          {
            "name": "zookeeper-1",
            "rbd": {
              "monitors": [
                "10.151.32.27:6789",
                "10.151.32.29:6789",
                "10.151.32.32:6789"
              ],
              "pool": "rbd",
              "image": "log-zookeeper-1",
              "user": "admin",
              "secretRef": {
                "name": "ceph-secret-default"
              },
              "fsType": "ext4",
              "readOnly": false
            }
          }
        ]
      }
    }
  }
}
```

Implementar el servicio zookeeper en el clúster kubernetes

Puedes usar este comando para implementar el servicio de zookeeper:

```
kubectl create -f zookeeper-svc-1.json
```

```
{
  "kind": "Service",
  "apiVersion": "v1",
  "metadata": {
    "name": "zookeeper-1",
    "labels": {
      "name": "zookeeper-1"
    }
  },
  "spec": {
    "ports": [
      {
        "name": "client",
        "port": 2181,
        "targetPort": 2181
      },
      {
        "name": "followers",
        "port": 2888,
        "targetPort": 2888
      },
      {
        "name": "election",
        "port": 3888,
```

```
        "targetPort": 3888
      }
    ],
    "selector": {
      "server-id": "1"
    }
  }
}
```

Grupo Zookeeper

Si desea obtener un grupo de zookeeper con 5 nodos, puede escribir los archivos zookeeper-rc-2/3/4 / 5.json y zookeeper-svc-2/3/4 / 5.json como se describe anteriormente y usar el comando kubectl para desplegarlos en el clúster kubernetes.

Lea Kubernetes en producción en línea:
https://riptutorial.com/es/kubernetes/topic/9153/kubernetes-en-produccion

Capítulo 3: Llamando a la API de Kubernetes

Examples

Usando Kubernetes Go Client - Fuera de Cluster

```
package main

import (
  "fmt"

  "k8s.io/client-go/1.5/kubernetes"
  "k8s.io/client-go/1.5/pkg/api/v1"
  "k8s.io/client-go/1.5/tools/clientcmd"
)

func main()  {
    config, err := clientcmd.BuildConfigFromFlags("", <kube-config-path>)
    if err != nil {
      return nil, err
    }

    c, err := kubernetes.NewForConfig(config)
    if err != nil {
      return nil, err
    }

    // Get Pod by name
    pod, err := c.Pods(v1.NamespaceDefault).Get("my-pod")
    if err != nil {
        fmt.Println(err)
        return
    }

    // Print its creation time
    fmt.Println(pod.GetCreationTimestamp())
}
```

Listar conjuntos de réplicas por implementación dada con kubernetes go client

```
package main

import (
    "k8s.io/kubernetes/pkg/api"
    unver "k8s.io/kubernetes/pkg/api/unversioned"
    "k8s.io/kubernetes/pkg/apis/extensions"
    "k8s.io/kubernetes/pkg/client/restclient"
    client "k8s.io/kubernetes/pkg/client/unversioned"
    "log"
    "os"
    "strings"
)
```

```
var logger *log.Logger

const (
    SERVER             string = "http://172.21.1.11:8080"
    RevisionAnnotation        = "deployment.kubernetes.io/revision"
)

func init() {
    logger = log.New(os.Stdout, "", 0)
}

func getReplicaSetsByDeployment(c *client.Client, deployment *extensions.Deployment)
([]extensions.ReplicaSet, error) {

    namespace := deployment.Namespace
    selector, err := unver.LabelSelectorAsSelector(deployment.Spec.Selector)
    if err != nil {
        return nil, err
    }
    options := api.ListOptions{LabelSelector: selector}
    rsList, err := c.Extensions().ReplicaSets(namespace).List(options)

    return rsList.Items, nil
}

func getDeploymentByReplicaSet(namespace string, c *client.Client, rs *extensions.ReplicaSet)
([]extensions.Deployment, error) {

    selector, err := unver.LabelSelectorAsSelector(rs.Spec.Selector)
    if err != nil {
        return nil, err
    }
    options := api.ListOptions{LabelSelector: selector}
    dps, err := c.Extensions().Deployments(namespace).List(options)
    if err != nil {
        return nil, err
    }

    return dps.Items, nil
}

func getDeploymentByReplicaSetName(namespace string, c *client.Client, rs
*extensions.ReplicaSet) (*extensions.Deployment, error) {

    name := rs.Name
    index := strings.LastIndex(name, "-")
    deploymentName := name[:index]

    dp, err := c.Extensions().Deployments(namespace).Get(deploymentName)
    if err != nil {
        return nil, err
    }

    return dp, nil
}

func main() {

    config := &restclient.Config{
        Host: SERVER,
    }
```

```
    c, err := client.New(config)
    if err != nil {
        logger.Fatalf("Could not connect to k8s api: err=%s\n", err)
    }

    list, err := c.Extensions().Deployments(api.NamespaceDefault).List(api.ListOptions{})
    if err != nil {
        logger.Fatalf("Could not list deployments: err=%s\n", err)
    }

    logger.Printf("Deployment -------> ReplicaSet: ")

    for _, deployment := range list.Items {
        rses, err := getReplicaSetsByDeployment(c, &deployment)
        if err != nil {
            logger.Fatalf("GetReplicaSetsByDeployment Error: err=%s\n", err)
        }

        for _, rs := range rses {
            logger.Printf("ReplicaSet assioated with Deployment: rs-name=%s, revision=%s, dp-
name=%s\n",
                rs.Name, rs.Annotations[RevisionAnnotation], deployment.Name)
        }
    }

    logger.Printf("\n\nReplicaSet -------> Deployment: ")
    rsList, err := c.Extensions().ReplicaSets(api.NamespaceDefault).List(api.ListOptions{})
    if err != nil {
        log.Fatalf("Could not list ReplicaSet")
    }

    for _, rs := range rsList.Items {
        dp, err := getDeploymentByReplicaSetName(api.NamespaceDefault, c, &rs)
        if err != nil {
            logger.Fatalf("GetDeploymentByReplicaSet Error: err=%s\n", err)
        }

        logger.Printf("Deployment assioated with ReplicaSet: rs-name=%s, revision=%s, dp-
name=%s\n",
            rs.Name, rs.Annotations[RevisionAnnotation], dp.Name)
    }
}
```

Retroceder con la revisión de conjuntos de réplicas usando kubernetes go client

```
package main

import (
    //"k8s.io/kubernetes/pkg/api"
    "k8s.io/kubernetes/pkg/client/restclient"
    "log"
    "os"
    // unver "k8s.io/kubernetes/pkg/api/unversioned"
    "k8s.io/kubernetes/pkg/apis/extensions"
    client "k8s.io/kubernetes/pkg/client/unversioned"
)
```

```
var logger *log.Logger

var annotations = map[string]string{
    "Image":                      "nginx:1.7.9",
    "UserId":                     "2",
    "kubernetes.io/change-cause": "version mismatch",
}

const (
    DEPLOYMENT         string = "nginx-test"
    REVERSION          int64  = 4
    SERVER             string = "http://172.21.1.11:8080"
    RevisionAnnotation        = "deployment.kubernetes.io/revision"
)

func init() {
    logger = log.New(os.Stdout, "", 0)
}

func rollBack(c *client.Client, dp *extensions.Deployment, revision int64) error {

    dr := new(extensions.DeploymentRollback)
    dr.Name = dp.Name
    dr.UpdatedAnnotations = annotations
    dr.RollbackTo = extensions.RollbackConfig{Revision: revision}

    // Rollback
    err := c.Extensions().Deployments("ops").Rollback(dr)
    if err != nil {
        logger.Printf("Deployment Rollback Error: err=%s\n", err)
        return err
    }

    return nil
}

func main() {
    config := &restclient.Config{
        Host: SERVER,
    }

    c, err := client.New(config)
    if err != nil {
        logger.Fatalf("Could not connect to k8s api: err=%s\n", err)
    }

    dp, err := c.Extensions().Deployments("ops").Get(DEPLOYMENT)
    if err != nil {
        logger.Fatalf("Could not list deployments: err=%s\n", err)
    }

    rollBack(c, dp, REVERSION)

}
```

Actualización continua con repliasets utilizando kubernetes go client

```
package main
```

```
import (
    //    "k8s.io/kubernetes/pkg/api"
    // unver "k8s.io/kubernetes/pkg/api/unversioned"
    "k8s.io/kubernetes/pkg/apis/extensions"
    "k8s.io/kubernetes/pkg/client/restclient"
    client "k8s.io/kubernetes/pkg/client/unversioned"
    "k8s.io/kubernetes/pkg/util/intstr"
    "log"
    "os"
)

var logger *log.Logger

const (
    //DEPLOYMENT           string = "nginx-deployment"
    DEPLOYMENT           string = "nginx-test"
    RevisionHistoryLimit int32  = 5
    SERVER               string = "http://172.21.1.11:8080"
    RevisionAnnotation   string = "deployment.kubernetes.io/revision"
)

func init() {
    logger = log.New(os.Stdout, "", 0)
}

func rollingUpdate(c *client.Client, dp *extensions.Deployment) error {

    // New a DeploymentStrategy
    ds := new(extensions.DeploymentStrategy)
    ds.Type = extensions.RollingUpdateDeploymentStrategyType
    ds.RollingUpdate = new(extensions.RollingUpdateDeployment)
    ds.RollingUpdate.MaxUnavailable = intstr.FromInt(int(dp.Spec.Replicas))

    dp.Spec.Strategy = *ds

    // Image
    //dp.Spec.Template.Spec.Containers[0].Image = "nginx:1.9.7"
    dp.Spec.Template.Spec.Containers[0].Image = "nginx:1.9"

    // Update
    //_, err := c.Extensions().Deployments(api.NamespaceDefault).Update(dp)
    _, err := c.Extensions().Deployments("ops").Update(dp)
    if err != nil {
        logger.Printf("Update Deployment Error: err=%s\n", err)
        return err
    }
    return nil
}

func main() {
    config := &restclient.Config{
        Host: SERVER,
    }

    c, err := client.New(config)
    if err != nil {
        logger.Fatalf("Could not connect to k8s api: err=%s\n", err)
    }

    //dp, err := c.Extensions().Deployments(api.NamespaceDefault).Get(DEPLOYMENT)
    dp, err := c.Extensions().Deployments("ops").Get(DEPLOYMENT)
```

```
    if err != nil {
        logger.Fatalf("Could not list deployments: err=%s\n", err)
    }

    rollingUpdate(c, dp)

}
```

Uso de Kubernetes Go Client - Dentro de Cluster

```
package main

import (
  "fmt"

  "k8s.io/client-go/1.5/kubernetes"
  "k8s.io/client-go/1.5/pkg/api/v1"
  "k8s.io/client-go/1.5/rest"
)

func main()  {
    config, err = rest.InClusterConfig()
    if err != nil {
      return nil, err
    }

    c, err := kubernetes.NewForConfig(config)
    if err != nil {
      return nil, err
    }

    // Get Pod by name
    pod, err := c.Pods(v1.NamespaceDefault).Get("my-pod")
    if err != nil {
        fmt.Println(err)
        return
    }

    // Print its creation time
    fmt.Println(pod.GetCreationTimestamp())
}
```

Lea Llamando a la API de Kubernetes en línea:
https://riptutorial.com/es/kubernetes/topic/5926/llamando-a-la-api-de-kubernetes

Creditos

S. No	Capítulos	Contributors
1	Empezando con kubernetes	cledoux, Community, Dimitris, idvoretskyi, jayantS, jkantihub, mohan08p, Ogre Psalm33, pagid, PumpkinSeed, webdizz
2	Kubernetes en producción	litanhua
3	Llamando a la API de Kubernetes	litanhua, Rush